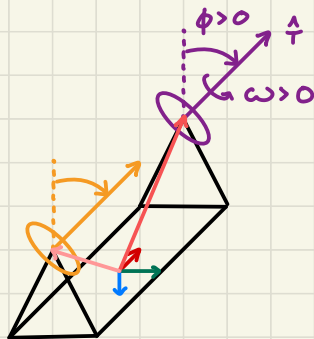


Rotor As wheel Derivation



$$H_{\text{rotor}} = \hat{T} \omega j$$

$$\dot{H}_{\text{rotor}} = \frac{d}{dt} \hat{T} \omega j + \hat{T} \omega j \times \mathbb{I} \omega^B$$

$$\downarrow$$

$$\dot{\hat{T}} \omega j + \hat{T} \dot{\omega} j + \hat{T} \omega j \times \mathbb{I} \omega^B$$

$$M_{\text{on rotor}} = j (\dot{\hat{T}} \omega + \hat{T} \dot{\omega} + \hat{T} \omega \times \mathbb{I} \omega^B)$$

$$\therefore M_{\text{on body}} = -j (\dot{\hat{T}} \omega + \hat{T} \dot{\omega} + \hat{T} \omega \times \mathbb{I} \omega^B)$$

$$\hat{T} = \begin{bmatrix} 0 \\ s\phi \\ -c\phi \end{bmatrix} \quad \therefore \dot{\hat{T}} = \begin{bmatrix} 0 \\ s\dot{\phi} \\ -c\dot{\phi} \end{bmatrix}$$

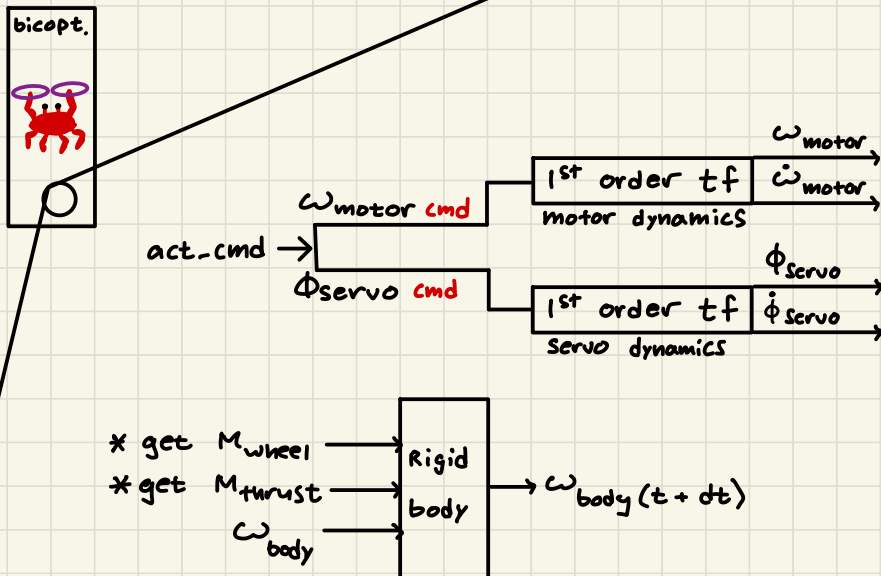
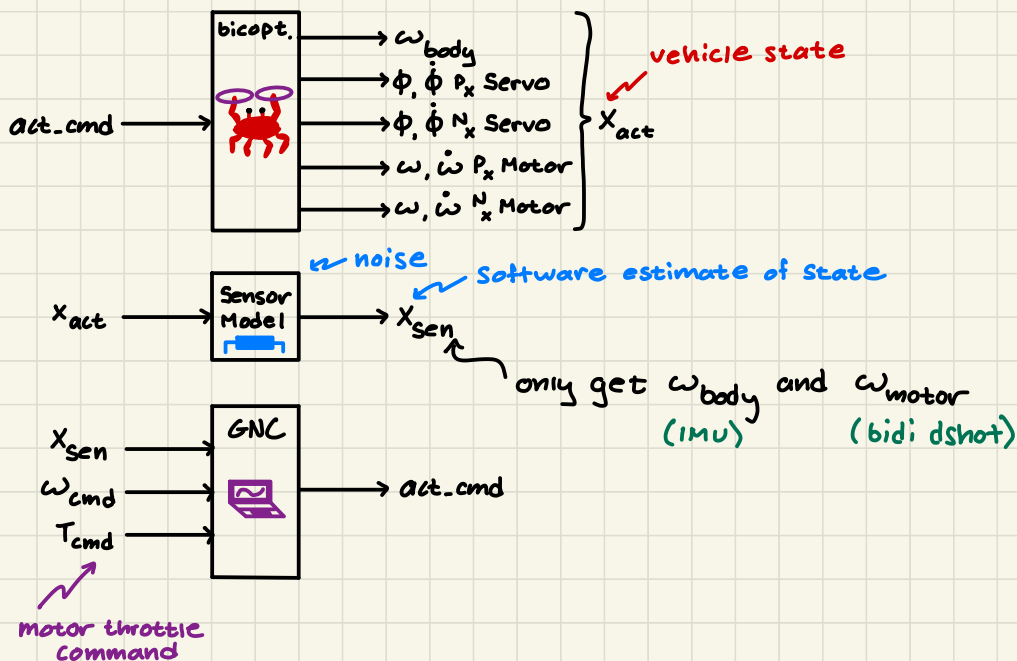
Assumptions

ϕ is only affected by the servos

ω is only affected by the motors

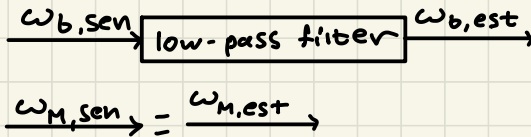
$$H_{\text{tilt}} \ll H_{\text{rotor}}$$

SIM Architecture planning





nonlinear



← we want to estimate this

$$M_{\text{on body}} = -j(\dot{T}\omega + \hat{T}\dot{\omega} + \hat{T}\omega \times {}^B\omega)$$

but,
 $M_{\text{on body}}(\phi, \dot{\phi}, \omega_M, \dot{\omega}_M, {}^B\omega)$
and
 $M_{\text{thrust}}(\phi, \omega, {}^B\omega)$

because the command is a function
of $E = \omega_{B, \text{des}} - \omega_{B, \text{est}}$

∴ coupling & instability

Idea: If we can predict what M_{wheel} will be
and pre-compensate, the loop is broken. (feed fwd)

Need:

$$\left. \begin{array}{l} \omega_b(t+dt) \\ \omega_m, \dot{\omega}_m(t+dt) \\ \phi, \dot{\phi}(t+dt) \end{array} \right\} \text{as a function of } u \text{ and current state}$$

↗
actuator
command

1) generate u as a function of $\omega_{b, \text{est}}$ and $\omega_{b, \text{des}}$

↖ estimate this

$$\dot{\omega} = \omega[A] + bu$$

Question: given what we know about the dynamics,
can we help the estimator out by giving it A (unknowns)
and only estimate unknowns rather than a whole 3×3 matrix?

Problem:

in reality...

$$\dot{\omega}^B = A' \dot{\omega} + B u + u_{\text{dist}} \quad \text{fcn of } I_1, I_2, I_3 = \text{fcn of } (\dot{\omega}^B, I_1, I_2, I_3)$$

$$\begin{bmatrix} \frac{1}{I_1} \\ \frac{1}{I_2} \\ \frac{1}{I_3} \end{bmatrix}$$

$$\begin{aligned} I_1 \dot{\omega}_1 + (I_3 - I_2) \omega_2 \omega_3 &= M_1 \\ I_2 \dot{\omega}_2 + (I_1 - I_3) \omega_3 \omega_1 &= M_2 \\ I_3 \dot{\omega}_3 + (I_2 - I_1) \omega_1 \omega_2 &= M_3 \end{aligned}$$

$$\rightarrow \cancel{I_1} - (I_3 - I_2) \omega_2 \omega_3$$

ignoring I_1

But maybe we can use this!

have a direct estimate

$$u_{\text{dist}} = -j \left(\hat{T} \omega + \hat{T} \dot{\omega} + \hat{T} \omega \times \dot{\omega} \right)$$

function of direct estimate

guess based on inputs

TLDL:

* inertias (constant)

* vehicle state (changing)

but...

$$f = -(A' \dot{\omega}^B + u_{\text{dist}}) \quad \text{but we don't care}$$

$$I \dot{\omega}^B = B u - f \quad (\text{Adrc})$$

$$f = B u - I \dot{\omega}^B \rightarrow f + I \dot{\omega}^B = B u \quad B^T (f + \dot{\omega}^B)$$

$$u = (I \dot{\omega}^B + f)^T$$

basically just
your control
gains

yucky numerical differentiation

1) estimate f

$$I \dot{\omega}^B_{\text{prev}} = \left(I \omega^B_{\text{EST}, t-1} - I \omega^B_{\text{EST}, t-2} \right) / dt_s$$

$$f = B u_{\text{prev}} - I \dot{\omega}^B_{\text{prev}}$$

2) compute u

$$I \dot{\omega}^B_{\text{curr}} = \left(I \omega^B_{\text{EST}, t-0} - I \omega^B_{\text{EST}, t-1} \right) / dt_s$$

$$u = B^T (f + I \dot{\omega}^B_{\text{curr}})$$

new idea: integrate everything

Question: how does the system evolve?

Answer:

$$\int \dot{\omega}^B = U + \dot{f}$$

input disturbance

what is $\dot{f}$?

$$\int \dot{\omega}^B = \int U + \dot{f}$$
$$\therefore \dot{f} = \int \dot{\omega}^B - \int U$$

ok; what if I want $\dot{\omega}^B$ to be $\dot{\omega}^B$

$$\dot{\omega}^B = \int_0^{t-1} U + \Delta \int_{t-1}^t U + \dot{f}(t)$$

$$\dot{\omega}^B - \int U - \dot{f}_{est}(t) = \dot{f}$$

$$\therefore U = \dot{\omega}^B - \dot{f}_{est}$$

numerical derivative of generalized disturbance estimate

numerical derivative of control input

okay let's uhhhh do it who needs gains

wrong! $\dot{f}$ is a function of $\dot{\omega}^B$ actual

but that is ~~open loop~~. let's have one more go

I think using $\int \dot{\omega}^B$ (numerical derivative) is ok but it should ~~not~~ be used to get $\dot{f}$ without some form of inherent numerical stability

1) say $\int \dot{\omega}^B = U + \dot{f}$

let's estimate $\dot{f}$ the same way we did before

$$\therefore \int \dot{\omega}^B - \int U = \dot{f}$$

2) let $\int \dot{\omega}^B$ be our desired state

if $\int \dot{\omega}^B_{actual} = \int U + \dot{f}$, then

$\dot{f}$ can be estimated as $\dot{\omega}^B_{actual} - \int U = \dot{f}$

note that $\dot{f}$ is not itself being integrated; but is instead re-computed every time.

$$\text{so, } \mathcal{I} \dot{\omega}^B = u + \dot{f}$$

$$\text{and } {}^1\omega^B = \int u + f$$

$$\text{we want } {}^1\omega^B - {}^1\omega^B = 0$$

so,

$$\int u + f - {}^1\omega^B = 0$$

$$\int_0^{t-1} u + \int_t^t u$$

$$\text{so, } \int_0^{t-1} u + \int_{t-1}^t u + f - {}^1\omega^B = 0$$

$$\text{so, } {}^1\omega^B =$$

wait! haha

pseudo-inverse allocator

$$M_{px} = \vec{r}_{cg2px} \times \hat{T}_f = \vec{r}_{cg2px} \times \begin{bmatrix} 0 \\ s\phi \\ -c\phi \end{bmatrix} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix} \begin{bmatrix} A_x \\ A_y \\ A_z \end{bmatrix} \times \begin{bmatrix} 0 \\ s\phi \\ -c\phi \end{bmatrix} \begin{bmatrix} B_x & 0 \\ B_y & t_y \\ B_z & t_z \end{bmatrix}$$

$$\therefore M_{px} = \begin{bmatrix} -r_y c\phi - r_z s\phi \\ r_z \textcircled{0} - (-r_x c\phi) \\ r_x s\phi - r_y \textcircled{0} \end{bmatrix} = \begin{bmatrix} -r_y c\phi - r_z s\phi \\ r_x c\phi \\ r_x s\phi \end{bmatrix}$$

$$M = \begin{bmatrix} -r_y c\phi_p - r_z s\phi_p & -r_y c\phi_n - r_z s\phi_n \\ r_x c\phi_p & r_x c\phi_n \\ r_x s\phi_p & r_x s\phi_n \end{bmatrix} \begin{bmatrix} f_{px} \\ f_{nx} \end{bmatrix}$$

(<=>)

$$f \begin{bmatrix} -r_y t_y - r_z t_z \\ r_x t_y \\ r_x t_z \end{bmatrix} \rightarrow \begin{bmatrix} -r_y, -r_z \\ r_x, 0 \\ 0, r_x \end{bmatrix} \begin{bmatrix} f t_y \\ f t_z \end{bmatrix}$$

pinv that. Then

$$\therefore M = \begin{bmatrix} -r_y, -r_z & -r_y, -r_z \\ r_x, 0 & r_x, 0 \\ 0, r_x & 0, r_x \end{bmatrix} \begin{bmatrix} f t_y \\ f t_z \\ f t_y \\ f t_z \end{bmatrix}$$

p_x n_x

$-c\phi$ $s\phi$

$$\phi = \text{atan2}(-f t_z, f t_y)$$

$$\therefore f s\phi = x \cdot s\phi$$

$$\therefore f = \frac{f s\phi}{s\phi}$$